Geekbrains

Создание приложения для заметок с регистрацией пользователей с использованием фреймворка Spring и Spring Security. Тестирование web-приложений с помощью фреймворков JUnit и Mockito
(Продолжение 2 года обучения)

Программа: Java разработчик

Специализация: java backend разработчик

Васина Анна Вадимовна

Г. Петрозаводск

2026

Введение 2
Глава 1. Теоретическая часть 3
1.1 О языке Java 4
1.2 Что такое фреймворки 4
1.3 Фреймворк спринг 6
1.4 Что такое Maven и зачем он нужен 8
1.5 Архитектура проекта
Глава 2. Теоретическая часть тестирования приложений
2.1 Понятие и цели тестирования
2.2 Виды тестирования
2.3 Инструменты и подходы в Java
Глава 3. Практическая часть
3.1 Структура моего проекта
3.2 Тестирование проекта
3.3 Как запустить программу
Заключение
Список литературы
Приложения

Введение

Мы живем в быстроменяющемся мире, когда запомнить все свои задачи и планы на день становится трудновыполнимой задачей; люди прибегают к разным способам фиксации своего расписания: использование блокнота, запись в сообщениях в социальной сети или использование специального приложения, заточенного под создание списка дел и расписания.

Перенос собственных планов из памяти мозга на физический носитель становится необходимостью для каждого, кто работает в мире, где планы меняются быстрее, чем их удается согласовать, и держать всю информацию исключительно в голове становится не только сложным делом, но и появляется риск что-нибудь упустить. В качестве дипломной работы я решила разработать собственное приложение, в котором пользователь сможет создавать заметки и составлять план действий.

Для выполнения этого проекта были поставлены следующие задачи:

1. Изучить фреймворк Spring
2. Изучить дополнительные инструменты, ускоряющие разработку программы, такие как: Spring Security, Lombok, Thymelef
3. Изучить, как работать со сборщиком проектов Maven

Глава 1. Теоретическая часть языка программирования Java

1.1 О языке Java

Java — один из самых популярных языков программирования в мире. Он был создан Джеймсом Гослингом и Патриком Ноттоном, сотрудниками компании Sun Microsystems, при поддержке Билла Джоя, сооснователя Sun Microsystems.

Компания Sun официально представила язык Java на конференции SunWorld 23 мая 1995 года. Затем, в 2009 году, компания Oracle купила компанию Sun, что объясняет, почему сейчас язык принадлежит Oracle.

Java описывается как многоцелевой, строго типизированный, объектно-ориентированный язык программирования. По замыслу, Java обладает минимальным количеством зависимостей от конкретных реализаций.

Этот язык программирования позволяет создавать приложения на множестве устройств. Его применение очень широкое, включая разработку программного обеспечения для мобильных устройств, POS-терминалов, банкоматов, Интернета вещей (IoT), а также веб-страниц.

Ключевые особенности языка Java:

- объектно-ориентированный язык программирования
- динамический язык программирования
- надёжность и безопасность
- переносимость и независимость от платформы
- высокая производительность

Благодаря своим отличным характеристикам, Java стала популярным и полезным языком программирования. Компания Sun охарактеризовала его как компилируемый и интерпретируемый

1.2 Что такое фреймворки

Согласно определению из Википедии,

«Фреймворк — программная платформа, определяющая структуру программной системы; программное обеспечение, облегчающее разработку и объединение разных компонентов большого программного проекта.»

Фреймворк отличается от библиотеки тем, что последняя может быть использована в программном продукте просто как набор подпрограмм похожей функциональности, не влияя на архитектуру программного продукта и не накладывая на неё никаких ограничений. В то время как фреймворк диктует правила построения архитектуры приложения, задавая на начальном этапе разработки поведение по умолчанию — «каркас», который нужно будет расширять и изменять согласно указанным требованиям

Иными словами и вкратце, можно дать следующее определение фреймворку:

«Фреймворк — готовая модель в IT, заготовка, шаблон для программной платформы, на основе которого можно дописать собственный код.»

У использования фреймворков есть ряд плюсов.

Во-первых, с точки зрения бизнеса разработка на фреймворке экономически эффективнее и качественнее по результату, нежели написание проекта без использования каких-либо платформ.

Во-вторых, приложения на фреймворках значительно проще сопровождать и дорабатывать. Дело в том, что их стандартизированная структура понятна всем разработчикам на этой платформе.

Фреймворки эффективны в проектах со сложной бизнес-логикой и высокими требованиями к скорости работы, надёжности и безопасности. В простых, типовых проектах лучше использовать другие решения.

Существенный минус фреймворков — открытый код большинства из них. Это значит, что нужно отдельно решать вопросы безопасности проекта.

Фреймворки дают стабильность и удобство разработки, но ограничивают программистов своей архитектурой.

Большинство фреймворков основано на шаблоне проектирования MVC (Model-View-Controller или модель-представление-контроллер).

Модель (Model) — это место, где хранятся данные приложения.

Представление или отображение (View) определяет взаимодействие с пользователем через модель. Он отвечает за отображение данных, определяет внешний вид проекта — веб-сайтов или приложений. От него зависят возможные действия пользователя: где печатать текст, какие кнопки нажимать.

Контроллер (Controller) отвечает за реакцию двух предыдущих объектов на действия пользователя. Его код и логика проверяют, как сайт обрабатывает запросы и выдает ли правильный результат.

1.3 Фреймворк спринг

Spring Framework (или коротко **Spring**) — универсальный фреймворк с открытым исходным кодом для Java-платформы

Spring обеспечивает решения многих задач, с которыми сталкиваются Java-разработчики и организации, которые хотят создать информационную систему, основанную на платформе Java. Из-за широкой функциональности трудно определить наиболее значимые структурные элементы, из которых он состоит. Spring не всецело связан с платформой Java Enterprise, несмотря на свою масштабную интеграцию с ней, что является важной причиной его популярности.

Spring может быть рассмотрен как коллекция меньших фреймворков или фреймворков во фреймворке. Большинство этих фреймворков может работать независимо друг от друга, однако они обеспечивают большую функциональность при совместном их использовании

Дополнительные библиотеки, которые были использованы в проекте:

- Lombok

  Проект Lombok позволяет сильно упростить Java-код, генерируя методы установки значений, генерации значений, hashCode, equals и ещё много чего.

  Добавляется в проект с помощью добавления зависимости в pom файл:

```
<dependency>
        <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  </dependency>
```

- Spring Security

  Spring Security это Java/Java EE фреймворк, предоставляющий механизмы построения систем аутентификации и авторизации, а также другие возможности обеспечения безопасности для промышленных приложений, созданных с помощью Spring Framework

  Добавляется в проект с помощью добавления зависимости в pom файл:

```
<dependency>
```

```
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
```

- Thymeleaf — это современный серверный шаблонизатор Java для веб— и автономных сред.

  Основная цель Thymeleaf — привнести элегантные естественные шаблоны в рабочий процесс разработки с помощью HTML, которые будут корректно отображаться в браузерах, а также работать как статические прототипы, обеспечивая упрощенное сотрудничество при командной работе.

  Благодаря модулям для фреймворка Spring, множеству интеграций с другими инструментами и возможности подключать собственные функции, Thymeleaf идеально подходит для современной веб—разработки на HTML5 вместе с джава.

  Добавляется в проект с помощью добавления зависимости в pom файл:

```
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring—boot—starter—thymeleaf</artifactId>
    </dependency>
```

- H2 database

  H2 — открытая кроссплатформенная СУБД, полностью написанная на языке Java.

  Добавляется в проект с помощью добавления зависимости в pom файл:

```
    <dependency>
        <groupId>com.h2database</groupId>
        <artifactId>h2</artifactId>
        <version>2.3.232</version>
    </dependency>
```

1.4 Что такое Maven и зачем он нужен

Maven — это мощный инструмент, который использует структуру проектов на основе XML. Он предоставляет широкий набор плагинов и позволяет разработчикам создавать сложные проекты с множеством зависимостей и настроек. Один из его основных принципов— «соглашение против конфигурации», что означает, что Maven предполагает наличие определенной структуры и настроек в вашем проекте, что упрощает работу с ним.

Помимо компиляции исходного кода, системы сборки выполняют множество дополнительных функций, таких как:

1. Управление зависимостями: Одна из самых важных функций систем сборки — это управление зависимостями между библиотеками и модулями вашего проекта. Maven позволяет автоматически загружать и интегрировать библиотеки из удаленных репозиториев, таких как Maven Central или JCenter.

2. Тестирование: Системы сборки интегрируются с различными фреймворками тестирования, такими как JUnit или TestNG, и автоматически запускают тесты во время сборки проекта. Это обеспечивает постоянное контролирование качества вашего кода.

3. Генерация документации: Maven может автоматически генерировать документацию для кода, используя такие инструменты, как Javadoc. Это значительно облегчает поддержку и развитие проекта на долгосрочной основе.

4. Пакетирование и дистрибуция: Системы сборки создают исполняемые файлы и архивы (например, JAR, WAR или EAR) для развертывания и распространения приложения. Это делает процесс дистрибуции быстрым и непринужденным.

5. Поддержка плагинов: Maven имеет мощную систему плагинов, которая позволяет расширять функциональность системы сборки путем добавления дополнительных задач и интеграции с другими инструментами, такими как системы контроля версий или инструменты для непрерывной интеграции.

- Структура проекта: XML—структура (файл pom.xml)
- Читабельность: менее читабельный из—за XML
- Скорость: стабильно, но медленно
- Гибкость: есть ограничения

Основные понятия Maven:

1. POM (Project Object Model):

POM — это XML—файл, который описывает проект и его настройки. Он является сердцем проекта Maven и включает информацию о зависимостях, плагинах, версиях и других параметрах.

2. Зависимости:

Зависимости — это внешние библиотеки или модули, необходимые для корректной работы вашего приложения. Maven автоматически управляет зависимостями, загружая их из удаленных репозиториев и интегрируя в проект.

3. Плагины:

Плагины — это компоненты, которые расширяют функциональность Maven, добавляя дополнительные задачи и интеграцию с другими инструментами.

4. Жизненный цикл:

Жизненный цикл — это последовательность фаз, определяющих процесс сборки проекта. Основные циклы жизни в Maven включают clean (очистка сборки), default (основная сборка) и site (генерация сайта проекта).

1.5 Архитектура проекта

Архитектура IT проекта — это процесс планирования структуры и взаимодействия компонентов программного обеспечения для достижения целей. Она включает в себя выбор технологий, определение требований, создание диаграмм и планирование тестирования.

Рассмотрим типичную архитектуру java-проекта:

src/main/java: Здесь хранится исходный код приложения.

src/main/resources: Здесь находятся ресурсы приложения, такие как файлы конфигурации, изображения ит. д.

src/test/java: Здесь размещается исходный код тестов приложения.

src/test/resources: Здесь хранятся ресурсы, связанные с тестированием.

pom.xml: Это основной файл конфигурации Maven, содержащий информацию о проекте и его настройках.

Рисунок 1. Структура проекта Maven

Глава 2. Теоретическая часть тестирования приложений

2.1 Понятие и цели тестирования

Тестирование программного обеспечения — это процесс проверки программы с целью выявления дефектов и оценки того, насколько результат её работы соответствует заданным требованиям и ожиданиям заказчика. Тестирование не доказывает полное отсутствие ошибок, но позволяет существенно снизить их количество и уменьшить вероятность критических сбоев при эксплуатации приложения.

Основная цель тестирования заключается в обнаружении дефектов на максимально ранних этапах жизненного цикла программного продукта. Чем раньше ошибка выявлена, тем дешевле и проще её исправить как с точки зрения трудозатрат разработчика, так и с точки зрения рисков для пользователя. Дополнительными целями тестирования являются повышение надежности и устойчивости программы к некорректным данным, проверка корректности реализованных функций, а также подтверждение того, что приложение удовлетворяет функциональным и нефункциональным требованиям.

Тестирование играет важную роль в обеспечении качества программного обеспечения. Оно позволяет убедиться, что реализованный функционал соответствует техническому заданию, бизнес-логика работает предсказуемо, а пользователь получает ожидаемый результат при нормальном и нестандартном использовании программы. Для учебных и итоговых проектов тестирование также демонстрирует умение автора не только разрабатывать код, но и критически оценивать его корректность и качество.

2.2 Виды тестирования

Существует множество классификаций видов тестирования, которые помогают структурировать процесс проверки программного продукта. Одной из наиболее распространённых является классификация по уровню, к которому относится проверяемый объект: от отдельных модулей до всей системы целиком.

К основным уровням тестирования относятся:

- ***Юнит-тестирование (модульное тестирование)*** — проверка отдельных модулей или классов программы в изоляции от остальных компонентов. На этом уровне тестируется, как правило, внутренняя логика отдельных методов, корректность обработки входных данных и формирование ожидаемого результата. Для приложений на Java юнит-тесты чаще всего пишутся с использованием специализированных фреймворков (например, JUnit), что позволяет автоматизировать запуск тестов и регулярно проверять работоспособность кода при внесении изменений.
- ***Интеграционное тестирование*** — проверка совместной работы нескольких модулей, сервисов или слоёв приложения. На этом этапе важна не только корректность отдельных компонентов, но и то, как они взаимодействуют между собой: например, как сервис обращается к базе данных, как контроллеры используют сервисный слой и т.п. Интеграционное тестирование помогает выявить ошибки на границах взаимодействия модулей, которые не видны на уровне отдельных классов.
- ***Системное тестирование*** — проверка уже сформированной системы в целом, как единого приложения. Здесь оценивается соответствие системы требованиям, проверяется функционирование всего набора возможностей с точки зрения конечного пользователя. В рамках системного тестирования рассматриваются реальные сценарии использования, которые включают полный путь обработки данных: от ввода до получения результата.

Ещё одна важная классификация связана со способом проведения тестирования:

- ***Ручное тестирование*** — когда тестировщик (или разработчик в учебном проекте) выполняет последовательность действий с приложением и фиксирует фактический результат. Такой подход часто используется для проверки пользовательского интерфейса, визуальных элементов, а также для первичной оценки корректности работы основных функций.
- ***Автоматизированное тестирование*** — когда проверки реализованы в виде тестовых сценариев, которые выполняются программно. Автоматические тесты позволяют быстро и регулярно перепроверять большую часть функционала, особенно после внесения изменений в код. Для Java-приложений автоматизацию обычно выполняют с помощью тестовых фреймворков, интегрированных в процесс сборки.

Также выделяют тестирование по цели проверки:

- ***Функциональное тестирование*** — направлено на проверку того, что реализованные функции соответствуют спецификации: каждый метод, экран или сценарий выполнения операции выполняет свою задачу и выдаёт ожидаемый результат.
- ***Нефункциональное тестирование*** — оценивает характеристики программы, не связанные напрямую с конкретными функциями: производительность, удобство использования, безопасность, устойчивость к ошибкам, потребление ресурсов и т.д.
- ***Регрессионное тестирование*** — проводится после внесения изменений в код, исправления ошибок или добавления нового функционала и направлено на проверку того, что ранее работавшие части системы не перестали работать корректно.

2.3 Инструменты и подходы в Java

В данном проекте тестирование реализовано с использованием стандартных инструментов экосистемы Java и фреймворка Spring. Основной акцент сделан на автоматизированных модульных и частично интеграционных тестах, что позволяет регулярно проверять корректность бизнес-логики и взаимодействия слоёв приложения при изменении кода.

Фреймворк JUnit

JUnit является де-факто стандартом для модульного тестирования в Java-проектах. Он предоставляет аннотации и инфраструктуру для описания, запуска и группировки тестов. Каждый тестовый метод в JUnit представляет собой отдельный сценарий проверки, который подготавливает входные данные, вызывает тестируемый метод и сравнивает фактический результат с ожидаемым с помощью утверждений (assertions).

Основные элементы JUnit, задействованные в проекте:

- Аннотация @Test для обозначения методов, которые являются тестами.
- Утверждения (assertEquals, assertTrue, assertFalse и т.п.) для проверки корректности результатов.
- Возможность группировать тесты по классам и пакетам, что упрощает структуру тестового набора и позволяет запускать отдельные группы тестов (например, только для сервисного слоя).

Использование JUnit интегрировано в процесс сборки Java-приложения: тесты запускаются автоматически при выполнении стандартных задач сборки (например, через Maven или Gradle). Это позволяет своевременно обнаруживать ошибки, возникающие после изменения кода, и служит основой для регрессионного тестирования.

Mockito и концепция мок-объектов

Для изоляции тестируемых компонентов от их внешних зависимостей в проекте применяется библиотека Mockito. Mockito реализует подход мок-объектов (mock objects) — специальных тестовых заменителей реальных зависимостей, которые имитируют их поведение. Такой подход позволяет:

- Проверять логику конкретного класса, не затрагивая реальную базу данных, внешние сервисы или другие сложные компоненты.
- Явно задавать ожидаемое поведение зависимостей (например, заранее определить, какой результат вернёт репозиторий при вызове определённого метода).
- Проверять факты взаимодействия: вызывались ли нужные методы, с какими параметрами и сколько раз.

В тестах для Spring-приложения мок-объекты обычно используются для подмены слоёв, которые не являются непосредственным объектом тестирования. Например, при тестировании сервисного слоя реальные репозитории, работающие с базой данных, подменяются моками. Это позволяет тестировать бизнес-логику сервиса независимо от состояния БД и скорости доступа к ней.

Глава 3. Практическая часть

3.1 Структура моего проекта

Структура моего проекта состоит из стандартных src/main/java, src/main/resources, в папке приложения com.example.noteapp содержатся папки:

- Config — для работы системы авторизации
- Controller — для обработки приема событий
- Entities — сущности приложения
- enum_entities — заранее определенные константы (для присвоения приоритета и категории)
- Repository — интерфейсы
- Serialization — сохранение данных в json-формате
- Service — для обработки логики

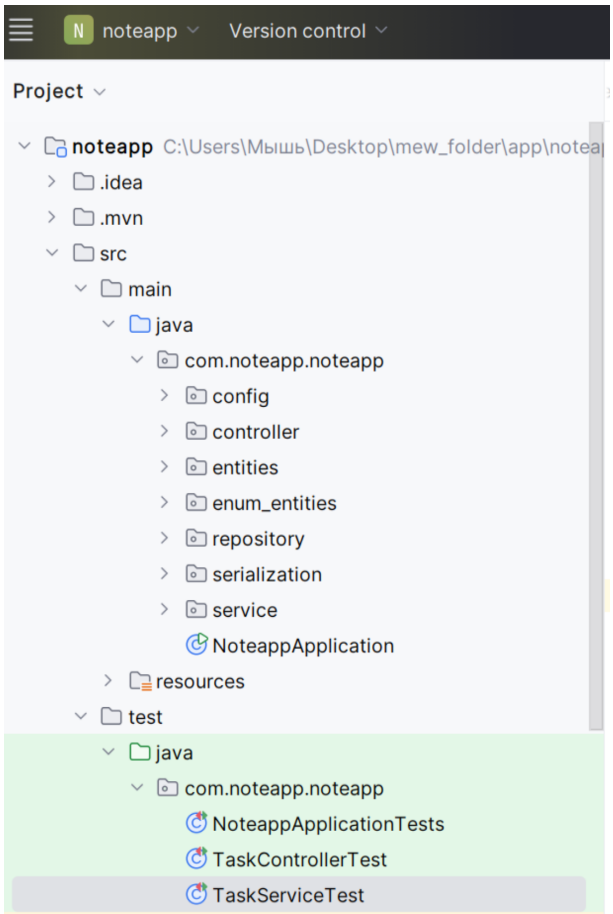

Рисунок 2. Скриншот структуры проекта

3.2 Тестирование проекта

Тестирование через чек-листы удобно тем, что позволяет формализовать ручную проверку и сделать её повторяемой. Чек-лист представляет собой перечень конкретных шагов и ожидаемых результатов, по которым можно быстро оценить работоспособность ключевых функций приложения. Такой подход снижает риск пропустить важный сценарий, упрощает фиксацию статусов проверок и даёт возможность другим участникам команды воспроизвести тесты по тем же шагам.

№	Функция	Шаги проверки	Ожидаемый результат	Статус (Passed / Failed)	Комментарий
1	Авторизация: валидный логин пользователя 1	1. Открыть экран входа. 2. Ввести корректные логин и пароль пользователя 1. 3. Нажать «Войти».	Пользователь 1 успешно авторизован, открывается главная страница/список заметок.	Passed	
2	Авторизация: валидный логин пользователя 2	1. Выйти из аккаунта, открыть экран входа. 2. Ввести корректные логин и пароль пользователя 2. 3. Нажать «Войти».	Пользователь 2 успешно авторизован, отображаются его заметки (заметки пользователя 1 не видны).	Passed	
3	Авторизация: неверный пароль	1. Открыть экран входа. 2. Ввести существующий логин и неверный пароль. 3. Нажать «Войти».	Авторизация отклонена, отображается сообщение об ошибке, вход не выполняется.	Passed	
4	Создание заметки	1.Авторизоваться.	Новая заметка	Passed	

№	Функция	Шаги проверки	Ожидаемый результат	Статус (Passed / Failed)	Коммента рий
		2. Нажать «Создать заметку». 3. Заполнить заголовок и текст. 4. Сохранить заметку.	появляется в списке, данные соответствуют введённым.		
5	Редактирова ние заметки	1.Авторизоваться. 2. Выбрать существующую заметку. 3. Нажать «Редактировать». 4. Изменить заголовок/текст. 5. Сохранить изменения.	В списке и при открытии заметки отображаются обновлённые данные.	Passed	
6	Удаление заметки	1.Авторизоваться. 2. Выбрать заметку. 3. Нажать «Удалить». 4. Подтвердить удаление	Заметка исчезает из списка, повторное открытие невозможно.	Passed	
7	Присвоение статуса важности	1.Авторизоваться. 2. Открыть заметку. 3. Установить статус «важная». 4. Сохранить.	Заметка помечена как важная (отображается соответствующий индикатор в списке/карточке).	Passed	
8	Присвоение статуса завершенност ти	1.Авторизоваться. 2. Открыть заметку. 3. Установить статус «завершена». 4. Сохранить.	Заметка помечена как завершённая (специальная иконка/отметка, при необходимости меняется визуальное оформление).	Passed	
9	Разделение данных по	1. Авторизоваться пользователем 1,	Пользователь 2 не видит заметки	Passed	

№	Функция	Шаги проверки	Ожидаемый результат	Статус (Passed / Failed)	Коммента рий
	пользователя м	создать/отметить несколько заметок. 2. Выйти. 3. Авторизоваться пользователем 2.	пользователя 1; создаёт и видит только свои заметки.		
10	Сериализаци я одиночной заметки в JSON	1. Создать или открыть существующую заметку. 2. Выполнить действие/операци ю, приводящую к сериализации заметки (сохранение/экспо рт).	Формируется корректное JSON-представлени е заметки: все ключевые поля (id, заголовок, текст, статусы) присутствуют и содержат ожидаемые значения.	Passed	
11	Сериализаци я списка заметок в JSON	1. Убедиться, что у пользователя есть несколько заметок. 2. Выполнить экспорт/сериализа цию списка.	Полученный JSON содержит массив объектов заметок, количество элементов соответствует количеству заметок в системе, структура каждого объекта корректна.	Passed	
12	Десериализа ция валидного JSON в объект	1. Взять заранее подготовленный валидный JSON заметки/списка заметок. 2. Запустить процедуру импорта/загрузки.	На основе JSON создаются корректные объекты заметок, все поля заполняются ожидаемыми значениями, заметки появляются в системе/списке.	Passed	
13	Перезапись существующ его файла	1. Выполнить экспорт заметок в JSON-файл. 2. Изменить/добавит	Содержимое файла обновляется и соответствует актуальному	Passed	

		ь заметку. 3. Повторно выполнить экспорт с тем же именем файла.	состоянию заметок, старые данные не сохраняются, ошибок записи не возникает.	
14	Чтение заметок из существующего файла JSON	1. Убедиться, что JSON-файл с заметками существует. 2. Запустить импорт/загрузку из файла.	Заметки из файла корректно загружаются в приложение: их количество и содержимое совпадают с данными в файле.	Passed
15	Обработка пустого или отсутствующего файла	1. При отсутствии файла система создает новый	Программа создает новый файл	Passed
16	Согласованность данных после цикла экспорт–импорт	1. Экспортировать текущие заметки в JSON-файл. 2. Очистить заметки в приложении (или использовать новый профиль/пользователя). 3. Импортировать данные из файла.	Множество заметок после импорта полностью совпадает с исходным: количество, заголовки, текст, статусы важности и завершенности.	Passed

3.3 Как запустить программу

Программа находится в репозитории на GitHub по ссылке https://github.com/KittyMew0/final_java_qa_work

После запуска программы на компьютере, будет предложено зарегистрировать нового пользователя, или воспользоваться предсозданным пользователем (username: testuser, password: password) с тестовой заметкой для изучения функционала.

Навигация внутри приложения может осуществляться с помощью указания путей:

- Главная страница и список заметок: / ;
- Добавление заметки: /notes/add ;
- Редактирование заметки: /notes/edit/{id}* ;
- Удаление заметки: /notes/delete/{id}* ;

*{id} - указание идентификатора заметки

Или с помощью кнопок, добавленных при использовании html и css.

Заключение

В процессе работы над проектом были изучены различные библиотеки и фреймворки, упрощающие разработку сервисных приложений, среди которых популярный фреймворк Spring и сборщик проектов Maven; задачи, поставленные в начале работы, были выполнены.

В теоретической части дипломной работы представлены ключевые определения и характеристики используемых библиотек и средств тестирования, в том числе подходы к модульному и интеграционному тестированию Java-приложений с использованием JUnit и Mockito. В практической части описаны структура проекта, взаимосвязи между классами, реализованные тесты для основных слоёв Spring-приложения, а также приведены примеры чек-листов для ручной проверки работоспособности веб-приложения.

В качестве итогового проекта было создано веб-приложение для создания заметок, корректность работы которого подтверждена с помощью автоматизированных и ручных тестов; в дальнейшем приложение может быть доработано и выпущено в публичный доступ.

Список литературы

Spring framework // springframework URL:

https://springframework.net/

Spring framework // Wikipedia URL:

https://en.wikipedia.org/wiki/Software_framework

H2 Database Engine // h2database URL:

https://www.h2database.com/html/main.html

Spring Security // Spring.io URL:

https://spring.io/projects/spring-security

Lombok // projectlombok URL:

https://projectlombok.org/

История Java // Baeldung URL:

https://www.baeldung.com/java-history

Приложения

Приложение 1. Класс AuthController

```
package com.noteapp.noteapp.config;

import com.noteapp.noteapp.entities.User;
import com.noteapp.noteapp.repository.UserRepository;
import jakarta.servlet.http.HttpSession;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;

/**
 * Controller for handling user authentication (signup and login).
 */
@Controller
public class AuthController {

    private final UserRepository userRepository;
    private final BCryptPasswordEncoder passwordEncoder = new
BCryptPasswordEncoder();

    public AuthController(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    /**
     * Registers a new user.
     *
     * @param username The username provided by the user.
     * @param password The plaintext password provided by the user.
     * @param model Spring Model for passing data to the view.
     * @return The signup page if registration fails, or redirects to the login page if
successful.
     */
    @PostMapping("/signup")
    public String registerUser(@RequestParam String username,
                               @RequestParam String password,
                               Model model) {
        if (userRepository.findByUsername(username).isPresent()) {
            model.addAttribute("error", "Username already exists.");
            return "signup";
        }

        String hashedPassword = passwordEncoder.encode(password);
        userRepository.save(new User(username, hashedPassword));

        return "redirect:/logininapp";
    }
```

```
    /**
     * Authenticates a user and establishes a session.
     *
     * @param username The username provided by the user.
     * @param password The plaintext password provided by the user.
     * @param session HttpSession to store the logged-in user.
     * @param model Spring Model for passing data to the view.
     * @return The login page if authentication fails, or redirects to the tasks page if
successful.
     */
    @PostMapping("/logininapp")
    public String loginUser(@RequestParam String username,
                            @RequestParam String password,
                            HttpSession session, Model model) {

        var userOpt = userRepository.findByUsername(username);

        if (userOpt.isPresent() && passwordEncoder.matches(password,
userOpt.get().getPassword())) {
            session.setAttribute("loggedInUser", userOpt.get());
            return "redirect:/tasks";
        }

        model.addAttribute("error", "Invalid username or password");
        return "logininapp";
    }
}
```

Приложение 2. Класс TaskController

```
package com.noteapp.noteapp.controller;

import com.noteapp.noteapp.entities.Task;
import com.noteapp.noteapp.service.TaskService;
import com.noteapp.noteapp.entities.User;
import com.noteapp.noteapp.enum_entities.Category;
import com.noteapp.noteapp.enum_entities.Priority;
import com.noteapp.noteapp.repository.TaskRepository;
import jakarta.servlet.http.HttpSession;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

import java.util.Optional;

/**
 * Controller for managing user tasks, including listing, adding, editing, completing, and
deleting tasks.
 * All endpoints are prefixed with "/tasks".
 */
@Controller
@RequestMapping("/tasks")
public class TaskController {

    private final TaskService taskService;
    private final TaskRepository taskRepository;

    public TaskController(TaskService taskService, TaskRepository taskRepository) {
        this.taskService = taskService;
        this.taskRepository = taskRepository;
    }

    /**
     * Displays the list of tasks for the logged-in user.
     *
     * @param model   Spring Model for passing data to the view.
     * @param session HttpSession to retrieve the logged-in user.
     * @return The task list page if the user is logged in, otherwise redirects to the login
page.
     */
    @GetMapping
    public String showTaskList(Model model, HttpSession session) {
        User user = (User) session.getAttribute("loggedInUser");

        if (user == null) {
            return "redirect:/logininapp";
        }

        model.addAttribute("tasks", taskRepository.findByUser(user));
        model.addAttribute("categories", Category.values());
        model.addAttribute("priorities", Priority.values());
```

```
        return "task-list";
    }

    /**
     * Adds a new task for the logged-in user.
     *
     * @param description The description of the task.
     * @param category    The category of the task.
     * @param priority    The priority of the task.
     * @param session     HttpSession to retrieve the logged-in user.
     * @return Redirects to the task list page.
     */
    @PostMapping("/add")
    public String addNewTask(@RequestParam String description,
                             @RequestParam Category category,
                             @RequestParam Priority priority,
                             HttpSession session) {

        User user = (User) session.getAttribute("loggedInUser");
        if (user == null) return "redirect:/logininapp";

        taskService.addTask(description, category, priority, user);
        return "redirect:/tasks";
    }

    /**
     * Marks a task as complete for the logged-in user.
     *
     * @param id      The ID of the task to mark as complete.
     * @param session HttpSession to retrieve the logged-in user.
     * @return Redirects to the task list page.
     */
    @PostMapping("/complete/{id}")
    public String completeTask(@PathVariable Long id, HttpSession session) {
        User user = (User) session.getAttribute("loggedInUser");
        Optional<Task> task = taskRepository.findById(id);

        if (user != null && task.isPresent() &&
task.get().getUser().getId().equals(user.getId())) {
            taskService.toggleComplete(id);
        }

        return "redirect:/tasks";
    }

    /**
     * Deletes a task for the logged-in user.
     *
     * @param id      The ID of the task to delete.
     * @param session HttpSession to retrieve the logged-in user.
     * @return Redirects to the task list page.
```

```
     */
    @PostMapping("/delete/{id}")
    public String deleteTask(@PathVariable Long id, HttpSession session) {
        User user = (User) session.getAttribute("loggedInUser");
        Optional<Task> task = taskRepository.findById(id);

        if (user != null && task.isPresent() &&
task.get().getUser().getId().equals(user.getId())) {
            taskService.deleteTask(id);
        }
        return "redirect:/tasks";
    }

    /**
     * Toggles the completion status of a task for the logged-in user.
     *
     * @param id      The ID of the task to toggle.
     * @param session HttpSession to retrieve the logged-in user.
     * @return Redirects to the task list page.
     */
    @PostMapping("/toggle/{id}")
    public String toggleTask(@PathVariable Long id, HttpSession session) {
        User user = (User) session.getAttribute("loggedInUser");

        Optional<Task> task = taskRepository.findById(id);
        if (user != null && task.isPresent() &&
task.get().getUser().getId().equals(user.getId())) {
            taskService.toggleComplete(id);
        }

        return "redirect:/tasks";
    }

    /**
     * Displays the edit form for a task.
     *
     * @param id      The ID of the task to edit.
     * @param model   Spring Model for passing data to the view.
     * @param session HttpSession to retrieve the logged-in user.
     * @return The edit task page if the user is authorized, otherwise redirects to the task
list page.
     */
    @GetMapping("/edit/{id}")
    public String showEditForm(@PathVariable Long id, Model model, HttpSession
session) {
        User user = (User) session.getAttribute("loggedInUser");
        Optional<Task> task = taskRepository.findById(id);

        if (user != null && task.isPresent() &&
task.get().getUser().getId().equals(user.getId())) {
            model.addAttribute("task", task.get());
            model.addAttribute("categories", Category.values());
```

```
            model.addAttribute("priorities", Priority.values());
            return "edit-task";
        }
        return "redirect:/tasks";
    }
}
```

Приложение 3. Класс Task

```
package com.noteapp.noteapp.entities;

import com.noteapp.noteapp.enum_entities.Category;
import com.noteapp.noteapp.enum_entities.Priority;
import jakarta.persistence.*;
import net.minidev.json.annotate.JsonIgnore;

import java.time.LocalDateTime;

/**
 * Represents a task entity in the application.
 * Each task has a description, category, priority, completion status, creation timestamp,
and an associated user.
 */
@Entity
public class Task {
    /**
     * Private fields: unique identifier for tasks,
     * description, category, priority of the task
     */
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String description;
    @Enumerated(EnumType.STRING)
    private Category category;
    @Enumerated(EnumType.STRING)
    private Priority priority;

    /**
     * Indicates whether the task is completed.
     * The timestamp when the task was created.
     */
    private boolean completed;
    private LocalDateTime createdAt;

    /**
     * The user who owns the task.
     */
    @ManyToOne
    @JoinColumn(name = "user_id")
    @JsonIgnore
    private User user;

    /**
     * Default constructor for JPA.
     */
    public Task() {}

    /**
     * Constructs a new Task with the specified details.
```

```
 *
 * @param id          The unique identifier for the task.
 * @param description The description of the task.
 * @param category    The category of the task.
 * @param priority    The priority of the task.
 */
public Task(Long id, String description, Category category, Priority priority) {
    this.id = id;
    this.description = description;
    this.category = category;
    this.priority = priority;
    this.completed = false;
    this.createdAt = LocalDateTime.now();
}

/**
 * Returns the CSS class for the task based on its completion status.
 *
 * @return "task-completed" if the task is completed, otherwise "task-active".
 */
public String getCssClass() {
    return completed ? "task-completed" : "task-active";
}

/** Getters and Setters
 * @return The unique identifier of the task.
 * @return The description of the task.
 * @return The category of the task.
 * @return The priority of the task.
 * @return The completion status of the task.
 * @return The timestamp when the task was created.
 */

public Long getId() { return id; }
public String getDescription() { return description; }
public Category getCategory() { return category; }
public Priority getPriority() { return priority; }
public boolean isCompleted() { return completed; }
public LocalDateTime getCreatedAt() { return createdAt; }

/**
 * @param id The unique identifier to set for the task.
 * @param description The description to set for the task.
 * @param category The category to set for the task.
 * @param priority The priority to set for the task.
 * @param completed The completion status to set for the task.
 * @param createdAt The timestamp to set for when the task was created.
 */
public void setId(Long id) { this.id = id; }
public void setDescription(String description) { this.description = description; }
public void setCategory(Category category) { this.category = category; }
public void setPriority(Priority priority) { this.priority = priority; }
```

```
    public void setCompleted(boolean completed) { this.completed = completed; }
    public void setCreatedAt(LocalDateTime createdAt) { this.createdAt = createdAt; }

    /**
     * @return The user who owns the task.
     */
    public User getUser() { return user; }

    /**
     * @param user The user to associate with the task.
     */
    public void setUser(User user) { this.user = user; }
}
```

Приложение 4. Класс User

```
package com.noteapp.noteapp.entities;

import jakarta.persistence.*;

import java.util.ArrayList;
import java.util.List;

/**
 * Represents a user entity in the application.
 * Each user has a unique username, password, and a list of associated tasks.
 */
@Entity
@Table(name = "users")
public class User {
    /**
     * The unique identifier for the user.
     */
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    /**
     * The username of the user. Must be unique and not null.
     */
    @Column(unique = true, nullable = false)
    private String username;

    /**
     * The password of the user. Must not be null.
     */
    @Column(nullable = false)
    private String password;

    /**
     * The list of tasks associated with the user.
     * Tasks are eagerly fetched and cascaded for all operations.
     */
    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL, fetch =
FetchType.EAGER)
    private List<Task> tasks = new ArrayList<>();

    /**
     * Default constructor for JPA.
     */
    public User() {}

    /**
     * Constructs a new User with the specified username and password.
     *
     * @param username The username of the user.
     * @param password The password of the user.
```

```
     */
    public User(String username, String password) {
        this.username = username;
        this.password = password;
    }
    /**
     * @return The unique identifier of the user.
     * @return The list of tasks associated with the user.
     * @return The username of the user.
     * @return The password of the user.
     */
    public Long getId() { return id; }
    public List<Task> getTasks() { return tasks; }
    public String getUsername() { return username; }
    public String getPassword() { return password; }

    /**
     * @param id The unique identifier to set for the user.
     * @param tasks The list of tasks to associate with the user.
     * @param username The username to set for the user.
     * @param password The password to set for the user.
     */
    public void setId(Long id) { this.id = id; }
    public void setTasks(List<Task> tasks) { this.tasks = tasks; }
    public void setUsername(String username) { this.username = username; }
    public void setPassword(String password) { this.password = password; }
}
```

Приложение 5. Класс Category

```
package com.noteapp.noteapp.enum_entities;

/**
 * Represents the possible categories for a task.
 * Each category helps classify the task based on its context or purpose.
 */
public enum Category {
    WORK,
    PERSONAL,
    HEALTH,
    SHOPPING,
    EDUCATION
}
```

Приложение 6. Класс Priority

```
package com.noteapp.noteapp.enum_entities;

/**
 * Represents the possible priority levels for a task.
 * Each priority level indicates the urgency or importance of the task.
 */
public enum Priority {
    LOW,
    MEDIUM,
    HIGH,
    URGENT
}
```

Приложение 7. Класс TaskRepository

```
package com.noteapp.noteapp.repository;

import com.noteapp.noteapp.entities.Task;
import com.noteapp.noteapp.entities.User;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.List;

/**
 * Repository interface for managing {@link Task} entities.
 * Provides methods for querying tasks, including finding tasks by user.
 */
@Repository
public interface TaskRepository extends JpaRepository<Task, Long> {

    /**
     * Finds all tasks associated with a specific user.
     *
     * @param user The user whose tasks are to be retrieved.
     * @return A list of tasks belonging to the specified user.
     */
    List<Task> findByUser(User user);
}
```

Приложение 8. Класс UserRepository

```
package com.noteapp.noteapp.repository;

import com.noteapp.noteapp.entities.User;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

/**
 * Repository interface for managing {@link User} entities.
 * Provides methods for querying users, including finding a user by username.
 */
public interface UserRepository extends JpaRepository<User, Long> {

    /**
     * Finds a user by their username.
     *
     * @param username The username of the user to find.
     * @return An {@link Optional} containing the user if found, otherwise empty.
     */
    Optional<User> findByUsername(String username);
}
```

Приложение 9. Класс AdminController

```
package com.noteapp.noteapp.serialization;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ResponseBody;

import java.io.IOException;

/**
 * Controller for handling administrative operations, such as data export.
 * Provides endpoints for administrative tasks, including exporting application data.
 */
@Controller
public class AdminController {
    private final DataExportService dataExportService;

    /**
     * Constructs an AdminController with the specified DataExportService.
     * @param dataExportService The service responsible for exporting data.
     */
    public AdminController(DataExportService dataExportService) {
        this.dataExportService = dataExportService;
    }

    /**
     * Triggers the export of application data to a JSON file.
     * The exported file is saved as "backup_data.json" in the project root.
     * @return A message indicating the success or failure of the export operation.
     */
    @GetMapping("/admin/export")
    @ResponseBody
    public String triggerExport() {
        try {
            dataExportService.exportDataToJson("backup_data.json");
            return "Backup created successfully in project root!";
        } catch (IOException e) {
            return "Export failed: " + e.getMessage();
        }
    }
}
```

Приложение 10. Класс DataExportService

```
package com.noteapp.noteapp.serialization;

import com.fasterxml.jackson.core.type.TypeReference;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.SerializationFeature;
import com.noteapp.noteapp.entities.Task;
import com.noteapp.noteapp.entities.User;
import com.noteapp.noteapp.repository.UserRepository;
import jakarta.annotation.PreDestroy;
import org.springframework.stereotype.Service;

import java.io.File;
import java.io.IOException;
import java.util.List;

/**
 * Service for exporting and importing application data to/from JSON files.
 * Handles serialization and deserialization of users and their associated tasks.
 */
@Service
public class DataExportService {

    private final UserRepository userRepository;
    private final ObjectMapper objectMapper;

    /**
     * Constructs a DataExportService with the specified UserRepository.
     * Initializes the ObjectMapper with pretty-printing enabled.
     *
     * @param userRepository The repository for accessing user data.
     */
    public DataExportService(UserRepository userRepository) {
        this.userRepository = userRepository;
        this.objectMapper = new ObjectMapper();
        this.objectMapper.enable(SerializationFeature.INDENT_OUTPUT);
    }

    /**
     * Exports all user data (including tasks) to a JSON file at the specified path.
     *
     * @param filePath The path where the JSON file will be saved.
     * @throws IOException If an I/O error occurs during file writing.
     */
    public void exportDataToJson(String filePath) throws IOException {
        List<User> allUsers = userRepository.findAll();
        objectMapper.writeValue(new File(filePath), allUsers);
        System.out.println("Data successfully exported to: " + filePath);
    }

    /**
     * Imports user data (including tasks) from a JSON file at the specified path.
```

```
     * If the file does not exist, logs a message and starts with an empty database.
     *
     * @param filePath The path from which the JSON file will be read.
     * @throws IOException If an I/O error occurs during file reading.
     */
    public void importDataFromJson(String filePath) throws IOException {
        File file = new File(filePath);

        if (!file.exists()) {
            System.out.println("No backup file found at " + filePath + ". Starting with an
empty database.");
            return;
        }

        List<User> users = objectMapper.readValue(file, new TypeReference<List<User>>()
{});

        for (User user : users) {
            if (user.getTasks() != null) {
                for (Task task : user.getTasks()) {
                    task.setUser(user);
                }
            }
        }

        userRepository.saveAll(users);
        System.out.println("Successfully imported " + users.size() + " users from
JSON!");
    }

    /**
     * Automatically exports all user data to a JSON file before the application shuts down.
     * This method is annotated with {@link PreDestroy} to ensure it runs on application
shutdown.
     */
    @PreDestroy
    public void onShutdown() {
        try {
            exportDataToJson("backup_data.json");
            System.out.println("Auto-saved data to JSON before shutdown.");
        } catch (IOException e) {
            System.err.println("Auto-save failed: " + e.getMessage());
        }
    }
}
```

Приложение 11. Класс DataInitializer

```
package com.noteapp.noteapp.serialization;

import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

/**
 * Component responsible for initializing application data on startup.
 * Implements {@link CommandLineRunner} to execute data import logic when the
application starts.
 */
@Component
public class DataInitializer implements CommandLineRunner {

    private final DataExportService dataExportService;

    /**
     * Constructs a DataInitializer with the specified DataExportService.
     * @param dataExportService The service responsible for importing data from JSON.
     */
    public DataInitializer(DataExportService dataExportService) {
        this.dataExportService = dataExportService;
    }

    /**
     * Imports data from a JSON file when the application starts.
     * This method is automatically called by Spring Boot on application startup.
     *
     * @param args Command-line arguments passed to the application.
     * @throws Exception If an error occurs during data import.
     */
    @Override
    public void run(String... args) throws Exception {
        dataExportService.importDataFromJson("backup_data.json");
    }
}
```

Приложение 12. Класс NoteappApplication

```
package com.noteapp.noteapp;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

/**
 * Main application class for the NoteApp Spring Boot application.
 * This class serves as the entry point for the application and enables auto-configuration.
 */
@SpringBootApplication
public class NoteappApplication {

	/**
	 * The main method that starts the Spring Boot application.
	 * @param args Command-line arguments passed to the application.
	 */
	public static void main(String[] args) {
		SpringApplication.run(NoteappApplication.class, args);
	}
}
```

Приложение 13. Класс NoteappApplicationTests

```
package com.noteapp.noteapp;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class NoteappApplicationTests {

	@Test
	void contextLoads() {
	}

}
```

Приложение 14. Класс TaskControllerTest

```
package com.noteapp.noteapp;

import com.noteapp.noteapp.entities.Task;
import com.noteapp.noteapp.entities.User;
import com.noteapp.noteapp.enum_entities.Category;
import com.noteapp.noteapp.enum_entities.Priority;
import com.noteapp.noteapp.repository.TaskRepository;
import com.noteapp.noteapp.repository.UserRepository;
import com.noteapp.noteapp.service.TaskService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.boot.webmvc.test.autoconfigure.AutoConfigureMockMvc;
import org.springframework.mock.web.MockHttpSession;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

import java.util.Optional;

import static org.mockito.Mockito.*;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static
org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

/**
 * Test class for the TaskController.
 * Uses Spring Boot's testing support and MockMvc to test controller endpoints.
 * Mocks repositories and services to isolate controller logic.
 */
@SpringBootTest
@AutoConfigureMockMvc
public class TaskControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @MockitoBean
    private TaskRepository taskRepository;

    @MockitoBean
    private TaskService taskService;

    @MockitoBean
    private UserRepository userRepository;

    /**
     * Tests that accessing the tasks endpoint without being logged in
     * results in a redirect to the login page.
     */
```

```
@Test
public void testAccessTasksWithoutLogin() throws Exception {
    mockMvc.perform(get("/tasks"))
            .andExpect(status().is3xxRedirection())
            .andExpect(redirectedUrl("/logininapp"));
}

/**
 * Tests that a user cannot delete a task owned by another user.
 * Verifies that the delete operation is not called for unauthorized users.
 */
@Test
public void testDeleteTaskAccessDenied() throws Exception {
    User owner = new User("owner", "pass");
    owner.setId(1L);

    User hacker = new User("hacker", "pass");
    hacker.setId(2L);

    Task task = new Task(99L, "Secret Task", Category.WORK, Priority.HIGH);
    task.setUser(owner);

    when(taskRepository.findById(99L)).thenReturn(Optional.of(task));

    MockHttpSession session = new MockHttpSession();
    session.setAttribute("loggedInUser", hacker);

    mockMvc.perform(post("/tasks/delete/99")
                .sessionAttr("loggedInUser", hacker))
            .andExpect(status().is3xxRedirection())
            .andExpect(redirectedUrl("/tasks"));

    verify(taskService, never()).deleteTask(99L);
}

/**
 * Tests that the task list endpoint shows only the tasks belonging to the logged-in user.
 * Verifies that the correct model attributes and view are returned.
 */
@Test
public void testShowTaskListShowsOnlyUserTasks() throws Exception {
    User currentUser = new User("bob", "password");

    mockMvc.perform(get("/tasks")
                .sessionAttr("loggedInUser", currentUser))
            .andExpect(status().isOk())
            .andExpect(model().attributeExists("tasks"))
            .andExpect(view().name("task-list"));

    verify(taskRepository).findByUser(currentUser);
}
```

```
/**
 * Tests that attempting to delete a non-existent task
 * results in a redirect and does not call the delete service.
 */
@Test
public void testDeleteNonExistentTask() throws Exception {
    User user = new User("bob", "pass");
    user.setId(1L);

    when(taskRepository.findById(999L)).thenReturn(Optional.empty());

    mockMvc.perform(post("/tasks/delete/999")
                .sessionAttr("loggedInUser", user))
            .andExpect(status().is3xxRedirection())
            .andExpect(redirectedUrl("/tasks"));

    verify(taskService, never()).deleteTask(999L);
}

/**
 * Tests that a user cannot access the edit form for a task owned by another user.
 * Verifies that unauthorized access results in a redirect.
 */
@Test
public void testShowEditFormUnauthorized() throws Exception {
    User owner = new User("owner", "pass");
    owner.setId(1L);
    User stranger = new User("stranger", "pass");
    stranger.setId(2L);

    Task task = new Task(10L, "Private Task", Category.PERSONAL, Priority.LOW);
    task.setUser(owner);

    when(taskRepository.findById(10L)).thenReturn(Optional.of(task));

    mockMvc.perform(get("/tasks/edit/10")
                .sessionAttr("loggedInUser", stranger))
            .andExpect(status().is3xxRedirection())
            .andExpect(redirectedUrl("/tasks"));
}

/**
 * Tests that a user can log in with a correctly encrypted password.
 * Verifies that successful login redirects to the tasks page.
 */
@Test
public void testLoginWithEncryptedPassword() throws Exception {
    BCryptPasswordEncoder encoder = new BCryptPasswordEncoder();
    String securePassword = encoder.encode("correct_pass");

    User user = new User("bob", securePassword);
```

```
        when(userRepository.findByUsername("bob")).thenReturn(Optional.of(user));

        mockMvc.perform(post("/logininapp")
                .param("username", "bob")
                .param("password", "correct_pass"))
            .andExpect(status().is3xxRedirection())
            .andExpect(redirectedUrl("/tasks"));
    }
}
```

Приложение 15. Класс TaskControllerTest

```
package com.noteapp.noteapp;

import com.noteapp.noteapp.entities.Task;
import com.noteapp.noteapp.entities.User;
import com.noteapp.noteapp.enum_entities.Category;
import com.noteapp.noteapp.enum_entities.Priority;
import com.noteapp.noteapp.repository.TaskRepository;
import com.noteapp.noteapp.service.TaskService;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;

import java.util.Optional;

import static org.junit.jupiter.api.Assertions.assertTrue;
import static org.mockito.Mockito.*;

/**
 * Test class for the TaskService.
 * Uses Mockito to mock dependencies and isolate the service logic for unit testing.
 */
@ExtendWith(MockitoExtension.class)
public class TaskServiceTest {

    @Mock
    private TaskRepository taskRepository;

    @InjectMocks
    private TaskService taskService;

    /**
     * Tests the addTask method of TaskService.
     * Verifies that the task is saved to the repository exactly once.
     */
    @Test
    public void testAddTask() {
        User user = new User("testUser", "password");
        String desc = "Buy Coffee";

        taskService.addTask(desc, Category.PERSONAL, Priority.MEDIUM, user);

        verify(taskRepository, times(1)).save(any(Task.class));
    }

    /**
     * Tests the toggleComplete method of TaskService.
     * Verifies that the task's completion status is toggled and saved to the repository.
     */
    @Test
```

```
    public void testToggleTaskCompletion() {
        Task task = new Task(1L, "Test Task", Category.WORK, Priority.HIGH);
        task.setCompleted(false);

        when(taskRepository.findById(1L)).thenReturn(Optional.of(task));

        taskService.toggleComplete(1L);

        assertTrue(task.isCompleted(), "Task should be true after toggle");
        verify(taskRepository).save(task);
    }
}
```

Приложение 16. CSS-файл для стилей

```
/* Folder styling */
.category-section {
    margin-top: 20px;
    border: 1px solid #ddd;
    border-radius: 8px;
    box-shadow: 2px 2px 5px rgba(0,0,0,0.05);
}

.category-header {
    background-color: #f8f9fa;
    padding: 12px;
    font-weight: bold;
    border-bottom: 1px solid #ddd;
    color: #555;
    border-radius: 8px 8px 0 0;
}

/* Shadowed effect for completed tasks */
.task-completed {
    background-color: #f9f9f9 !important;
    opacity: 0.6;
}

.task-completed span {
    text-decoration: line-through;
    color: #888;
}

/* Priority Colors */
.priority-HIGH { color: #d9534f; font-weight: bold; }
.priority-MEDIUM { color: #f0ad4e; }
.priority-LOW { color: #5cb85c; }
```

Приложение 17. HTML-файл edit-task.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
   <title>Edit Task</title>
   <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
<body>
<div class="container">
   <h2>Edit Task</h2>
   <form th:action="@{/tasks/add}" method="post"> <input type="hidden" name="id"
th:value="${task.id}">
      <input type="text" name="description" th:value="${task.description}" required>

      <select name="category">
         <option th:each="cat : ${categories}" th:value="${cat}"
               th:text="${cat}" th:selected="${cat == task.category}"></option>
      </select>

      <select name="priority">
         <option th:each="prio : ${priorities}" th:value="${prio}"
               th:text="${prio}" th:selected="${prio == task.priority}"></option>
      </select>

      <button type="submit">Save Changes</button>
      <a th:href="@{/tasks}">Cancel</a>
   </form>
</div>
</body>
</html>
```

Приложение 18. HTML-файл logininapp.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Login - To-Do App</title>
    <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
<body>
<div class="container">
    <h2>Login</h2>
    <p th:if="${error}" style="color: red;" th:text="${error}"></p>

    <form th:action="@{/login}" method="post">
        <div class="form-group">
            <label>Username:</label>
            <input type="text" name="username" required>
        </div>
        <div class="form-group">
            <label>Password:</label>
            <input type="password" name="password" required>
        </div>
        <button type="submit">Login</button>
    </form>
    <p>Don't have an account? <a th:href="@{/signup}">Register here</a></p>
</div>
</body>
</html>
```

Приложение 19. HTML-файл signup.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Register - To-Do App</title>
    <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
<body>
<div class="container">
    <h2>Create an Account</h2>
    <p th:if="${error}" style="color: red;" th:text="${error}"></p>
    <form th:action="@{/register}" method="post">
        <div class="form-group">
            <label>Username:</label>
            <input type="text" name="username" required>
        </div>
        <div class="form-group">
            <label>Password:</label>
            <input type="password" name="password" required>
        </div>
        <button type="submit">Register</button>
    </form>
    <p>Already have an account? <a th:href="@{/logininapp}">Login here</a></p>
</div>
</body>
</html>
```

Приложение 20. HTML-файл task-list.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
   <title>My To-Do App</title>
   <link rel="stylesheet" th:href="@{/css/style.css}">
</head>
<body>
<div class="container">
   <div style="display: flex; justify-content: space-between; align-items: center;">
      <p>Welcome, <strong
th:text="${session.loggedInUser.username}">User</strong>!</p>
      <a th:href="@{/logout}"><button style="background-color: #f44336; color: white;
border: none; padding: 5px 10px; cursor: pointer;">Logout</button></a>
   </div>

   <div class="status-bar" th:with="total=${tasks.size()},
completedCount=${tasks.stream().filter(t -> t.completed).count()}">
      <p>
         Progress: <span th:text="${completedCount}">0</span> / <span
th:text="${total}">0</span> tasks done
      </p>
      <div style="background-color: #eee; border-radius: 10px; height: 10px; width:
100%;">
         <div th:style="'background-color: #4CAF50; height: 100%; border-radius: 10px;
width: ' +
                  (${total} > 0 ? (${completedCount} * 100 / ${total}) : 0) + '%'">
         </div>
      </div>
   </div>

   <h1>My Tasks</h1>

   <form th:action="@{/tasks/add}" method="post" style="margin-bottom: 20px;">
      <input type="text" name="description" placeholder="New task..." required>
      <select name="category">
         <option th:each="cat : ${categories}" th:value="${cat}" th:text="${cat}"></option>
      </select>
      <select name="priority">
         <option th:each="prio : ${priorities}" th:value="${prio}" th:text="${prio}"></option>
      </select>
      <button type="submit">Add Task</button>
   </form>

   <hr>

   <div th:each="cat : ${categories}"
       th:if="${tasks.stream().anyMatch(t -> t.category == cat)}"
       class="category-section">
      <div class="category-header" th:text="${cat}">Category Name</div>

      <div th:each="task : ${tasks}" th:if="${task.category == cat}"
```

```
            th:classappend="${task.completed} ? 'task-completed' : "" class="task-item">

            <div>
                <span th:text="${task.description}"></span>
                <small th:class="'priority-' + ${task.priority}" th:text="'[' + ${task.priority} +
']'"></small>
            </div>

            <div class="actions">
                <form th:action="@{/tasks/toggle/{id}(id=${task.id})}" method="post"
style="display:inline;">
                    <button type="submit" th:text="${task.completed} ? 'Undo' : 'Done'"></button>
                </form>
                <a th:href="@{/tasks/edit/{id}(id=${task.id})}"><button>Edit</button></a>
                <form th:action="@{/tasks/delete/{id}(id=${task.id})}" method="post"
style="display:inline;">
                    <button type="submit" style="color:red;">X</button>
                </form>
            </div>
        </div>
    </div>
</div>
</body>
</html>
```

Приложение 21. Файл apllication.properties

```
spring.application.name=noteapp
spring.datasource.username=sa
spring.datasource.password=password
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.thymeleaf.prefix=classpath:/templates/
spring.thymeleaf.suffix=.html
spring.thymeleaf.cache=false
spring.web.resources.static-locations=classpath:/static/
spring.datasource.url=jdbc:h2:file:./data/todolistdb
spring.datasource.driverClassName=org.h2.Driver
spring.jpa.hibernate.ddl-auto=update
```

Приложение 22. Файл pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
                 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
        <modelVersion>4.0.0</modelVersion>
        <parent>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-starter-parent</artifactId>
                <version>3.4.0</version> <relativePath/>
        </parent>

        <groupId>com.noteapp</groupId>
        <artifactId>noteapp-V2</artifactId>
        <version>0.0.1-SNAPSHOT</version>
        <name>NoteAppV2</name>

        <properties>
                <java.version>17</java.version>
        </properties>

        <dependencies>
                <dependency>
                        <groupId>org.springframework.boot</groupId>
                        <artifactId>spring-boot-starter-web</artifactId>
                </dependency>
                <dependency>
                        <groupId>org.springframework.boot</groupId>
                        <artifactId>spring-boot-starter-thymeleaf</artifactId>
                </dependency>

                <dependency>
                        <groupId>org.springframework.boot</groupId>
                        <artifactId>spring-boot-starter-data-jpa</artifactId>
                </dependency>
                <dependency>
                        <groupId>com.h2database</groupId>
                        <artifactId>h2</artifactId>
                        <scope>runtime</scope>
                </dependency>
                <dependency>
                        <groupId>org.springframework.boot</groupId>
                        <artifactId>spring-boot-starter-security</artifactId>
                </dependency>
                <dependency>
                        <groupId>org.thymeleaf.extras</groupId>
                        <artifactId>thymeleaf-extras-springsecurity6</artifactId>
                </dependency>

                <dependency>
                        <groupId>com.fasterxml.jackson.core</groupId>
```

```
            <artifactId>jackson-databind</artifactId>
        </dependency>
        <dependency>
            <groupId>commons-io</groupId>
            <artifactId>commons-io</artifactId>
            <version>2.18.0</version>
        </dependency>
        <dependency>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
            <optional>true</optional>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <optional>true</optional>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>org.springframework.security</groupId>
            <artifactId>spring-security-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
                <configuration>
                    <excludes>
                        <exclude>
                            <groupId>org.projectlombok</groupId>
                            <artifactId>lombok</artifactId>
                        </exclude>
                    </excludes>
                </configuration>
            </plugin>
        </plugins>
    </build>
</project>
```